La récursité

1) Pour chacun des cas suivants, décrivez l'effet d'un appel de **`mystere(5);`**

```
public static void mystere(int n) {
    if(n >= 1) {
        mystere(n - 1);
        System.out.println(n);
    }
}
```

```
public static void mystere(int n) {
    if (n >= 1) {
        System.out.println(n);
        mystere(n - 1);
    }
}
```

```
public static void mystere(int n) {
   if (n >= 1) {
       System.out.println(n);
       mystere(n + 1);
   }
}
```

```
public static void mystere(int n) {
   if (n >= 1) {
       mystere(n + 1);
       System.out.println(n);
   }
}
```

2) Ecrivez une fonction récursive qui renvoie la factorielle d'un nombre entier positif.

3) Le n^{ème} nombre de *Fibonacci* est défini de la manière suivante : Si n > 2 : $F_n = F_{n-1} + F_{n-2}$ avec $F_1 = 1$ et $F_2 = 1$. Ecrivez une fonction récursive qui renvoie la valeur du n^{ème} nombre de *Fibonacci*. Développez l'exécution de `fib(5);`. Combien d'additions doit effectuer votre algorithme pour arriver au résultat?

4) Soient *a* un entier strictement positif, écrivez une fonction récursive qui calcule a^n pour *n* positif ou nul. Pour rappel : a^0 vaut 1. Combien de multiplication doit effectuer votre algorithme pour calculer 2^{24} ?

5) Servez-vous du fait que, par exemple, $x^{24} = (x * x)^{12}$ pour diminuer dramatiquement le nombre de multiplications et d'appels récursifs dans la solution de l'exercice précédent. Combien de multiplication doit effectuer votre algorithme pour calculer 2^{24} ? Combien en faut-il pour 2^{1000} ? Comparez avec celui de la question précédente.

6) Ecrivez une fonction (récursive, bien sûr) qui calcule le nombre de chiffres nécessaires pour écrire un nombre entier positif ou nul. Par exemple, le nombre deux mille sept cent vingt-trois s'écrit à l'aide de 4 chiffres.

7) Ecrivez une fonction qui calcule la somme des chiffres d'un nombre entier positif ou nul.

8) Ecrivez une fonction qui calcule la somme des entiers d'un `Array<Integer>`.

9) Nous avons vu précédemment la fonction de recherche dichotomique dans un tableau trié. Écrivez une fonction récursive qui détermine, par recherche dichotomique, si un entier apparaît dans un tableau trié d'entiers. Utilisez une fonction auxiliaire.

10) Ecrivez une fonction qui calcule le nombre de chiffre 2 dans un nombre entier positif ou nul.

11) Ecrivez une fonction qui détermine si tous les chiffres d'un nombre entier positif ou nul sont pairs.

12) Ecrivez une fonction qui affiche un nombre entier positif ou nul à l'envers.

13) Ecrivez une fonction qui détermine le plus grand des chiffres d'un nombre entier positif ou nul.

14) Ecrivez une fonction qui renvoie la valeur décimale d'un nombre binaire non signé. Par exemple, si l'on passe l'entier 1101 à la fonction, celle-ci renvoie 13 (1 + 4 + 8).

15) Ecrivez une fonction qui détermine le plus grand des entiers d'un `Array<Integer>`.

16) Ecrivez une fonction qui détermine le nombre de majuscules d'un `String`.